

CSC1018 Logic

Introduction to Prolog

David Sinclair

CSC1018 David Sinclair ©2024

Overview

Prolog is a programming language that is suited to symbolic, non-numeric computations.

A Prolog program consists of:

- clauses;
- facts;
- rules; and
- procedures.

Prolog is a *declarative* programming language.

imperative Describes a program in terms of the steps that change the program's state (values held by the program's variables).

declarative Describes the program in term of the properties of the solution.

Prolog's built-in backtracking engine searches for a solution consistent with the properties described.

Facts

Prolog works by defining relations between symbols and then submitting one or more queries to the Prolog run-time system to determine for which values of the query variables are the defined relations true.

Facts can be written as relations. In Prolog, to state the fact that Tom is Bob's parent, we write:

```
parent ( tom , bob ).
```

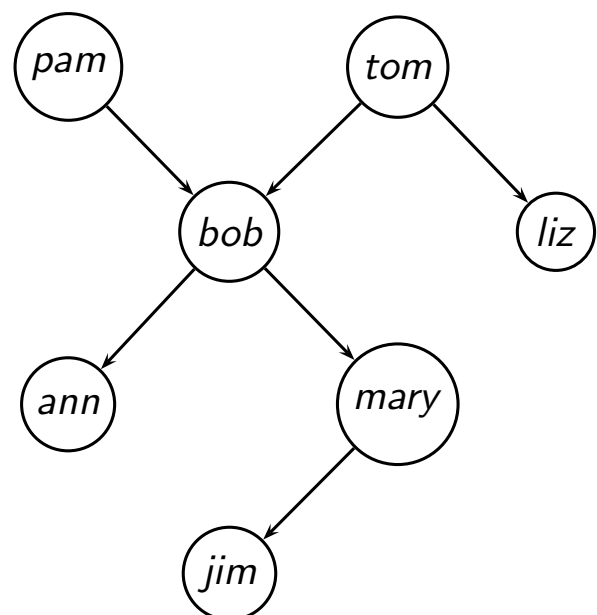
This expresses:

- a relation called *parent*,
- that takes two arguments, *tom* and *bob*,
- where *tom* and *bob* are symbols since they begin with a lower case letter (a-z).

Facts (2)

The following Prolog program describes a particular family tree.

```
parent ( pam , bob ).  
parent ( tom , bob ).  
parent ( tom , liz ).  
parent ( bob , ann ).  
parent ( bob , mary ).  
parent ( mary , jim ).
```



Querying Facts

We can query the previous program as follows.

```
?- parent (tom , bob ).  
true .
```

since Prolog can find this asserted fact in the program.

However,

```
?- parent (ann , jim ).  
false .
```

We can replace a symbol in a query by a variable. Variables start with an uppercase character.

```
?- parent (X, liz ).  
X = tom .
```

Querying Facts (2)

Some queries, such as who are Bob's children, may have multiple answers.

```
?- parent (bob , C ).  
C = ann
```

By typing the semi-colon (;) character we can get the next possible solution

```
?- parent (bob , C ).  
C = ann ;  
C = mary .
```

Querying Facts (3)

Or we can ask:

```
?- parent(X, Y).
X = pam ,
Y = bob ;
X = tom ,
Y = bob ;
X = tom ,
Y = liz ;
...
```

We can terminate the sequence of answers by pressing the return key.

Querying Facts (4)

We can construct more complicated queries by combining simpler queries.

Who are Jim's grandparents?

```
?- parent(Y, jim), parent(X,Y).
Y = mary ,
X = bob ;
false .
```

Which X is the parent of Y given that Y is a parent of Jim?

Who are Tom's grandchildren?

```
?- parent(tom, X), parent(X,Y).
X = bob ,
Y = ann ;
X = bob ,
Y = mary ;
false .
```

Querying Facts (5)

Things to note about a Prolog program.

- A Prolog program consists of a series of *clauses*, each ending in a period.
- A *clause* can define a *relations* by simply stating the n-tuple of objects that satisfy the *relation*.
- A user can easily *query* the Prolog run-time system about the relations defined in the program.
- The arguments of a *query* can be concrete objects or constants, called *atoms*, or general objects, called *variables*.
- A query to the Prolog run-time system is a series of *goals*. The query `parent(tom,X), parent(X, Y)` is satisfied if both *goals*, `parent(tom,X)` and `parent(X, Y)`, are satisfied.
- If a *goal* is satisfied then it has *succeeded*, otherwise it has *failed*.
- If multiple answers satisfy the goals, then the Prolog run-time system will find as many as the user requests.

Rules

We can specify new relations by using *rules*. A *fact* is something that is unconditionally **true**, whereas a *rule* is **true** only if some condition is **true** about other *rules* and *facts*.

We can define an child rules as follows.

```
child(X, Y) :- parent(Y, X)
```

A *rule* consists of:

- A condition (the right-hand side of the rule), called the *body*.
- A consequence (the left-hand side of the rule), called the *head*.
- If all the *goals* of the *body* of a rule succeed, the the *head* of the rule succeeds.
- Variables in rules are assumed to be universally quantified and can be read as 'for all'. `hasChild(X) :- parent(X, Y)` can be read as "for all X and Y, if X is the parent of Y then X has a child".

Rules (2)

Let's extend our family Prolog program by adding facts about the gender of the people in the family.

```
female(pam).
```

```
male(tom).
```

```
male(bob).
```

```
female(liz).
```

```
female(mary).
```

```
female(ann).
```

```
male(jim).
```

and adding the following rules.

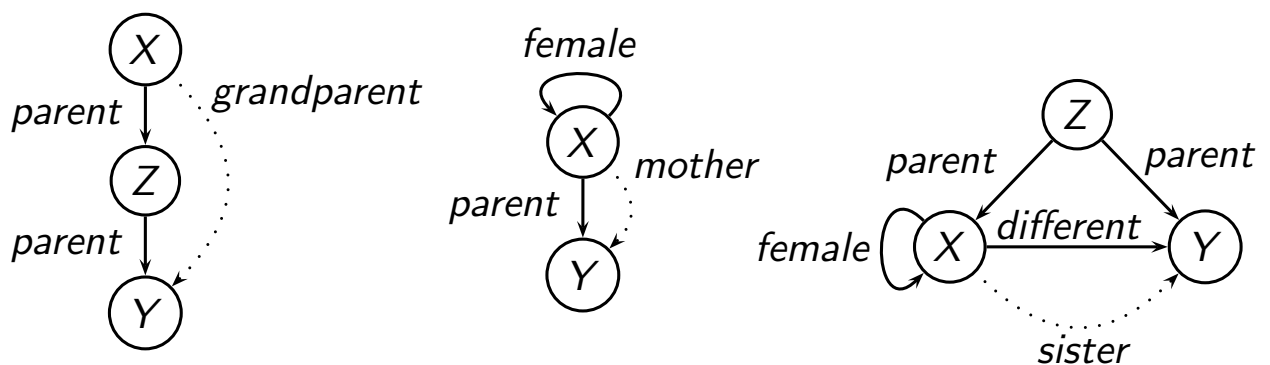
```
grandparent(X,Y) :- parent(X,Z), parent(Z,Y).
```

```
mother(X,Y) :- female(X), parent(X,Y).
```

```
sister(X,Y) :- female(X), parent(Z,X),  
parent(Z,Y), different(X,Y).
```

CSC1018 David Sinclair ©2024

Rules (3)



Recursive Rules

Rules can be *recursive*, i.e. the rule's definition depends on the rule itself.

Recursive rules have 2 components:

- **A base case** that ensures the rule terminates.
- **A recursive case** that describes how the rule applied to the current argument depends on the rule applied to previous values of the argument.

Consider the following rule for the predecessor relation.

```
predecessor(X,Y) :- parent(X,Y).
```

```
predecessor(X,Y) :- parent(X,Z), predecessor(Z,Y).
```

How does Prolog answer a query?

Consider the following Prolog program fragment.

```
parent(pam, bob).
parent(tom, bob).
parent(tom, liz).
parent(bob, ann).
parent(bob, mary).
parent(mary, jim).
```

```
predecessor(X,Y) :- parent(X,Y).           % rule pr1
```

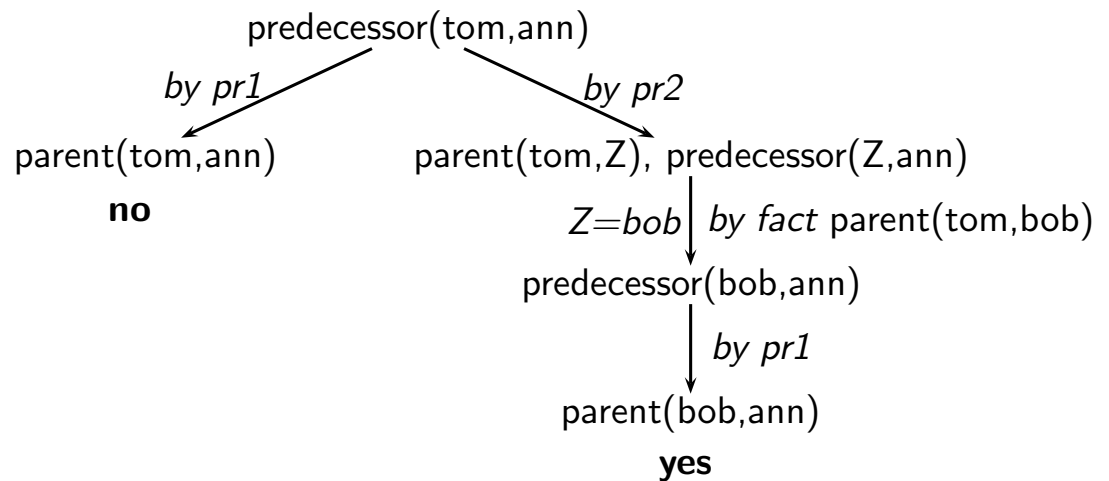
```
predecessor(X,Y) :- parent(X,Z),           % rule pr2
                      predecessor(Z,Y).
```

and the following query

```
?- predecessor(tom, ann).
```

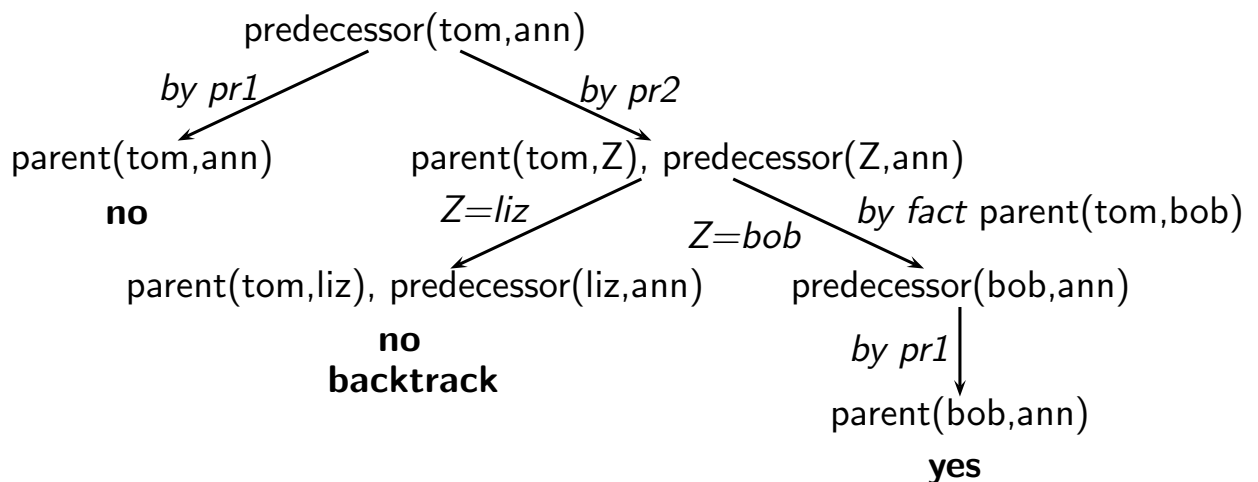
How does Prolog answer a query? (2)

- The Prolog run-time system will search the clauses from start to end in order to find a clause where the head “matches” the current goal.
- If the current goal cannot be satisfied the next “matching” clause is tried.



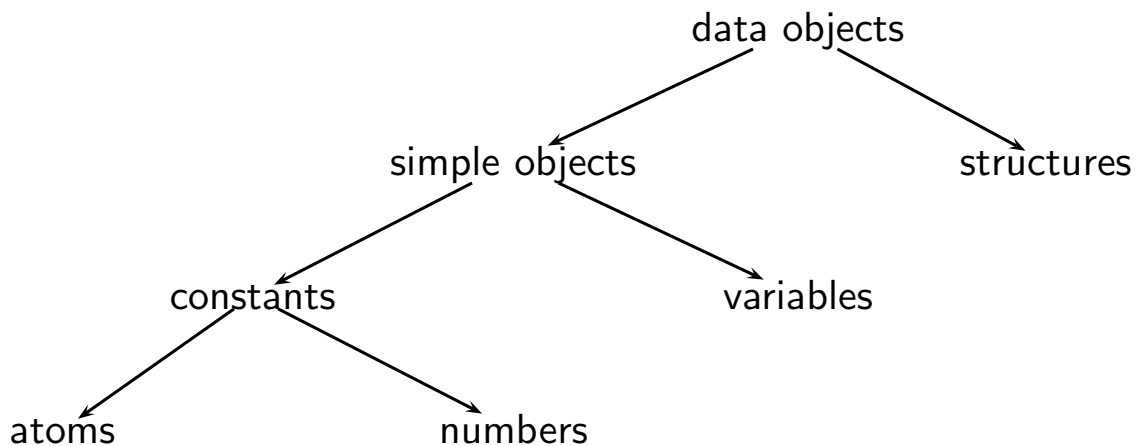
How does Prolog answer a query? (3)

What happens if we swap the order of clauses so that `parent(tom, liz)` is before `parent(tom, bob)`?



Data Objects in Prolog

In Prolog data objects are classified as:



Atoms and Numbers

Atoms

1. Strings of letters, digits ('0'-'9') or underscore ('_') characters, starting with a lower-case character, e.g. tom, nil, c14, alpha_bound, lowerBound.
2. Strings composed of special characters (+-*/<>=:.&_). Some sequences, such as :- are already reserved by Prolog.
3. Strings enclosed in single quotes, e.g. 'Tom', 'Mr_Smith'

Numbers can be integers or floating-point numbers.

- 12 and -123 are example of integers.
- 0.123 and 10.987 are examples of floating-point numbers.
Since Prolog is designed for symbolic computation, it is best to avoid heavy numeric computation.

Variables

A *variable* is a string of letters, digits or underscore characters that start with either an upper-case character or an underscore, e.g. `X`, `StartTime`, `Finishing_time`, `_answer`.

If a *variable* only occurs once in a clause, then it can be replaced by the *anonymous variable*, `'_'`, e.g. `haschild(X) :- parent(X,Y)` is equivalent to `haschild(X) :- parent(X,_)`. Each `'_'` in a clause represents a separate *anonymous variable*.

The *scope* of a variable is that clause in which it is mentioned. Therefore the variable `Student` in two different clauses are not the same variable.

Structures

Structures are objects that have several components, that in turn, can be other structures or variables. Each structure is composed of:

- a *functor* that names the structure; and
- a set of components.

A structure suitable for recording the date could be `date(1,may,2012)`. Here the *functor* is `date` and the components represent the day, month and year of the date.

To represent a triangle on a 2-Dimensional (2-D) plane we can define a structure `triangle` that uses another structure `point`.

```
triangle(point(1,2), point(6,3), point(4,5))
```

Structures (2)

To represent a triangle in 3-Dimensional (3-D) space we need to use a 3-D point, e.g. `point3D(1,2,3)` and not a 2-D point as above.

```
triangle ( point3D ( 1 , 2 , 3 ) , point3D ( 6 , 3 , 7 ) ,
           point3D ( 4 , 5 , 8 ) )
```

However, we need not use a new functor `point3D` as we can still use the name `point`, e.g. `point(1,2,3)` because a functor is defined by:

- its name, which is an atom, and;
- its *arity*, which is the number of its components/arguments.

```
triangle ( point ( 1 , 2 , 3 ) , point ( 6 , 3 , 7 ) , point ( 4 , 5 , 8 ) )
```

Matching

When Prolog encounters the expression

```
date ( D , M , 2012 ) = date ( D1 , may , Y1 )
```

it finds the *most general* instantiation of the variables to satisfy the expression. In this case it would be $D=D1$, $M=may$ and $Y1=2012$. This is more general than $D=12$, $D1=12$, $M=may$, $Y1=2012$.

If this expression is part of a larger expression then choosing the *most general* match gives the Prolog run-time a better chance of finding a match to the overall expression, e.g.

```
date ( D , M , 2012 ) = date ( D1 , may , Y1 ) ,
date ( D , M , 2012 ) = date ( 21 , M , Y )
```

Matching (2)

The general rules for matching two terms X and Y are:

- If X and Y are constants the X and Y match only if they are the same object.
- If X is a variable and Y is anything then X and Y match and X is instantiated to Y . If Y is a variable the Y is instantiated to X .
- If X and Y are structures then they match only if
 - X and Y have the same principle functor; and
 - all their corresponding components match.

The matching of the components determines the values each component is instantiated to.

The Declarative Meaning of a Prolog program

Given a goal G and a Prolog program P , the declarative meaning says that G is true if and only if there is a clause C in P such that:

- there is clause instance I of C such that
 - the head of I is identical to G , and
 - all the goals in the body of I are true.

If the goals are separated by a comma, e.g. $Q :- R, S, T$, then the list of goals is true if all goals in the list are true for the *same instantiation*. A comma denotes *conjunction* of the goals.

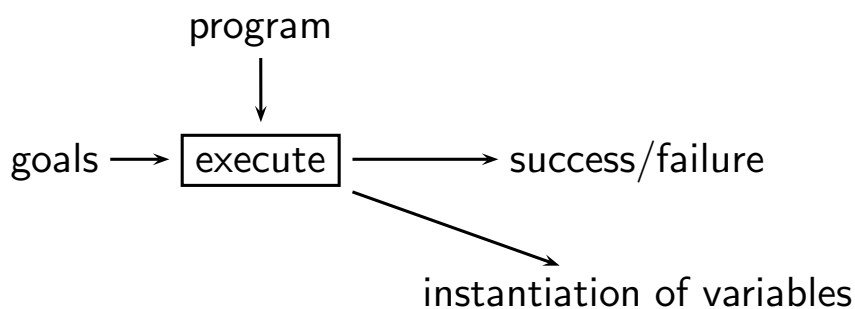
If the goals are separated by a semicolon, e.g. $Q :- R; S$, then the list of goals is true if any of goals in the list is true. A semicolon denotes *disjunction* of the goals. $Q :- R; S$ is equivalent to

$Q :- R.$

$Q :- S.$

The Procedural Meaning of a Prolog program

A Prolog program also has a procedural meaning. This describes the process the Prolog run-time system uses to determine if a set of goals are satisfied. Understanding the procedural meaning is very important in understanding how to improve the efficiency a Prolog program.



The Procedural Meaning of a Prolog program (2)

Given a goal list $G_1, G_2, \dots, G_m$ and a Prolog program P , **execute** operates as follows:

- If goal list is empty, return *success*.
- Search P from top to bottom for a clause C ($H :- B_1, \dots, B_n$) whose head H matches the first goal in the goal list.
 - If there is no such clause C , return *failure*.
 - Otherwise, rename the variables of C so that C and $G_1, G_2, \dots, G_m$ have no common variables, giving C' ($H' :- B_1', \dots, B_n'$). Match H' and G_1 let the resulting variable instantiation by S .
- Replace the first goal in the list by the body of C' , giving a new goal list $B_1', \dots, B_n', G_2, \dots, G_m$.
- Apply the variable instantiation S , resulting in the goal list $B_1'', \dots, B_n'', G_2', \dots, G_m'$.
- Apply **execute** to the new goal list.

The Procedural Meaning of a Prolog program (3)

```
big(bear).           % clause 1
big(elephant).       % clause 2
small(cat).          % clause 3

brown(bear).         % clause 4
black(cat).          % clause 5
grey(elephant).      % clause 6

dark(X) :- black(X). % clause 7

dark(X) :- brown(X). % clause 8
```

The Procedural Meaning of a Prolog program (4)

Consider the query
dark(X), big(X).

1. Scanning the program from top to bottom to find a clause that matches dark(X), we find clause 7, giving a new goal list black(X),big(X).
2. Scanning from top to bottom to find a match for black(X) we find clause 5, giving X=cat and a goal list big(cat).
3. Scanning from top to bottom for a match to big(cat) does not find a match. So we undo the variable instantiation X=cat and continue scanning from after clause 5 with goal list black(X),big(X).
4. Since there is no match for black(X) we undo the match dark(X)=black(X) and continuing scanning.

The Procedural Meaning of a Prolog program (5)

5. Scanning from clause 7 with goal list `dark(X), big(X)` finds a match between `dark(X)` and clause 8. The resulting goal list is `brown(X),big(X)`.
6. Scanning from top to bottom to find a match for `brown(X)` we find clause 4, giving `X=bear` and a goal list `big(bear)`.
7. Scanning from top to bottom to find a match for `big(bear)` we find clause 1, giving a empty goal list.
8. Since the goal list is empty, return *success*.

Order of clauses and goals

Consider the following 4 variations of the predecessor relation which simply reorder the clauses and/or goals.

% original version

```
pred1(X,Y) :- parent(X,Y).
pred1(X,Y) :- parent(X,Z), pred1(Z,Y).
```

% swap order of clauses in original

```
pred2(X,Y) :- parent(X,Z), pred1(Z,Y).
pred2(X,Y) :- parent(X,Y).
```

% swap order of goals in 2nd clause of original version

```
pred3(X,Y) :- parent(X,Y).
pred3(X,Y) :- pred3(X,Z), parent(Z,Y).
```

% swap order of goals and clauses in original version

```
pred4(X,Y) :- pred4(X,Z), parent(Z,Y).
pred4(X,Y) :- parent(X,Y).
```

Order of clauses and goals (2)

While all 4 versions are declaratively equivalent, they are **not** procedurally equivalent!

- `pred1` always works.
- `pred2` always works, but tends to take longer.
- `pred3` sometimes works.
- `pred4` never works as it get stuck in an infinite loop.

From this we can deduce some general guidelines for writing Prolog programs.

- It is generally better to try the simplest idea first.
- Check base cases first.

Lists

A *list* is a sequence of any number of Prolog atoms.
e.g. `[tom, football, golf, harry, poker]`

In Prolog, any structured data is represented internally as a tree.
So, the “list-like” appearance above is just an external representation of the list.

A list has 2 forms:

- It is either *empty*, `[ ]`, or
- it consists of:
 - a *head*, which is the 1st item in the list; and
 - a *tail*, which is the rest of the list.

The *head* and *tail* are combined by a special functor,

`.( Head, Tail)`

Lists (2)

Our example list from the previous slide would be represented internally as

```
.( tom, .( football, .( golf, .( harry, .( poker, [ ] ) ) ) ) ) )
```

Both notation (`.( )`) and (`[ ]`) can be used to represent lists. Generally, we prefer the square bracket notation.

Lists can contain other lists, eg.

```
?- Hobbies1 = [ football , golf ] ,
   Hobbies2 = [ poker ] ,
   L = [ tom , Hobbies1 , harry , Hobbies2 ] .
Hobbies1 = [ football , golf ] ,
Hobbies2 = [ poker ] ,
L = [ tom , [ football , golf ] , harry , [ poker ] ] .
```

Lists (3)

We usually refer to the *tail* of a list as a single object and Prolog provide a special operator, vertical bar, that represents the remainder of a list.

```
L = [ a | Tail ]
```

Any number of atoms can be listed before the vertical bar. So the following are all equivalent.

```
[ a | [ b , c ] ] = [ a , b | [ c ] ] = [ a , b , c | [ ] ]
```

The **key** thing to remember about a list is that it is a *recursive* data structure.

List Operations - membership

Prolog has a number of built-in predicates for manipulating lists. Let's see how these could be implemented.

Remembering that a list is recursive, X is an element of a list L if:

1. X is the *head* of the list; or
2. X is an element of the *tail* of the list.

In Prolog these two facts can be stated as:

```
member(X, [X|_]).
```

```
member(X, [_|Tail]) :-  
    member(X, Tail).
```

Remember that `'_'` is the *anonymous variable*.

List Operation - Concatenation

We will define a relation,

```
conc(L1, L2, L3)
```

that will return true if $L3$ is the concatenation of $L1$ and $L2$, and false otherwise.

```
conc([a,b], [c,d,e], [a,b,c,d,e])
```

returns true whereas

```
conc([a,b], [c,d,e], [c,d,a,b])
```

would return false.

Again, there are two cases:

1. $L1$ is empty, in which case $L2$ and $L3$ must be the same; and
2. the heads of $L1$ and $L3$ must be the same and tail of $L3$ must be the tail of $L1$ concatenated with $L2$.

List Operation - Concatenation (2)

In Prolog, we represent these 2 facts as:

```
conc([ ], L, L).
```

```
conc([X|L1], L2, [X|L3]) :-  
    conc(L1, L2, L3).
```

Now we can execute some queries.

```
?- conc([a,b], [c,d,e], [a,b,c,d,e]).  
true
```

```
?- conc([a,b], [c,d,e], L).  
L = [a,b,c,d,e].
```

```
?- conc(L, [d,e], [a,b,c,d,e]).  
L = [a,b,c] ;  
false.
```

List Operation - Concatenation (3)

```
?- conc(L1, L2, [a,b,c,d]).  
L1 = [ ],  
L2 = [a,b,c,d] ;  
L1 = [a],  
L2 = [b,c,d] ;  
L1 = [a,b],  
L2 = [c,d] ;  
L1 = [a,b,c],  
L2 = [d] ;  
L1 = [a,b,c,d],  
L2 = [ ] ;  
false.
```

```
?- conc(_, [B,apr,A], [jan,feb,mar,apr,may,jun,jul]).  
B = mar,  
A = may ;  
false.
```

List Operation - Adding and Deleting an Element

Adding an element:

```
add(X, L, [X|L]).
```

Deleting an element has 2 cases:

- the item to be deleted is the head of the list, or
- the item to be deleted is in the tail.

```
delete(X, [X|L], L).
```

```
delete(X, [Y|Tail], [Y|Tail1]) :-  
    delete(X, Tail, Tail1).
```

delete is non-deterministic.

```
insert(X, List, BiggerList) :-  
    delete(X, BiggerList, List).
```

List Operation - Sublist

S is a sublist of L if:

- L can be split into two lists L1 and L2; and
- L2 can be split into two lists S and L3.

```
sublist(S, L) :-  
    conc(L1, L2, L),  
    conc(S, L3, L2).
```

List Operation - Permutations

A *permutation* of a list is a reordering of the elements in the list.

We will define a relation that is **true** if the second list is a reordering of the first list and is defined by considering two cases of the first list.

- If the first list is empty, then the second list must also be empty.
- If the first list has the form $[X|L]$, then permute L to form a new list $L1$ and then insert X into $L1$.

`permutation([], []).`

```
permutation([X|L], P) :-
    permutation(L, L1),
    insert(X, L1, P).
```

Note: You need to be very careful using permutation. What could be the problem with `permutation(L, [a,b,c])??`

Operator Notation

Operators can be:

- *infix*, $a + b$;
- *prefix*, $+(a, b)$; and
- *postfix*, $a b +$.

While *infix* notation that we generally use in mathematics, Prolog internally uses *prefix* notation, where the operator is the **functor** of the relation, and the operands are the arguments of the relation.

Prolog supports *infix* and *postfix* notations as external representations, but internally still uses *prefix* notation.

Operator Notation (2)

Consider the expression

$$a + b * c.$$

This could be read as:

- $+(a, *(b,c))$; or
- $*(+(a,b), c)$.

The correct reading of the expression depends on the *precedence* and *type* of the operators.

Prolog has special clauses called *directives*. The **op** directive has the following syntax:

$\text{:- op}(\textit{precedence}, \textit{type}, \textit{name}).$

Operator Notation (3)

The operator's *name* is an atom and becomes the **functor** of the operator.

The *precedence* of the operator is usually between 0 and 1200 (implementation specific). Operators with *lower precedence* bind **more tightly** than operators with higher precedences.

If an argument is in parenthesis or is an unstructured data object its precedence is 0. If the argument is a structured data object then its precedence is the precedence of its principal functor.

The *type* of an operator can be:

- *infix*: $x\textit{fx}$, $x\textit{fy}$, $y\textit{fx}$
- *prefix*: $\textit{fx}$, $\textit{fy}$
- *postfix*: $\textit{xf}$, $\textit{yf}$

Operator Notation (4)

What is the difference between **xfx**, **xfy** and **yfx**?

The **f** represents the operator.

The **x** represents an argument whose precedence must be less than the the precedence of the operator.

The **y** represents an argument whose precedence is less than or equal to the the precedence of the operator.

yfx represents *left-to-right* evaluation.

Operator Notation (5)

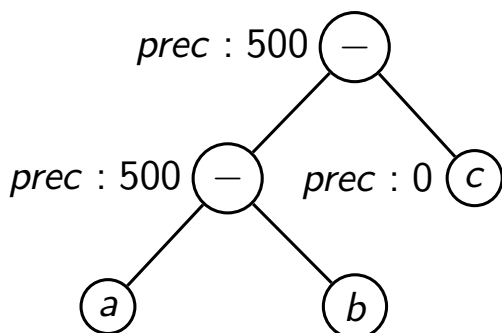
Consider

$$a - b - c$$

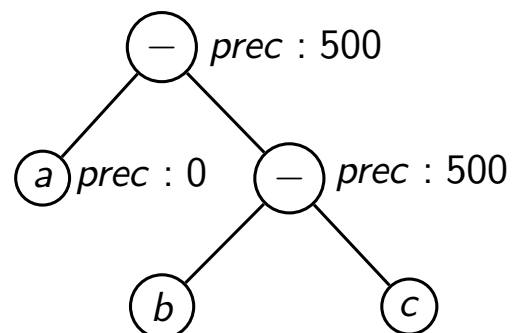
which should be interpreted as $(a - b) - c$ and not $a - (b - c)$.

Then, minus should be defined as

$\text{:- op}(500, \text{yfx}, -)$.



This matches **yfx**.



This does not match **yfx**.

Operator Notation (6)

We could write one of DeMorgan's laws as:

`equivalence(not(and(A,B)), or(not(A), not(B))).`

Or as:

`:- op(800, xfx, <===>).`

`:- op(700, xfy, v).`

`:- op(600, xfy, &).`

`:- op(500, fxy, ~).`

`~(A & B) <===> ~A v ~B.`

Arithmetic

Prolog is a symbolic language so

`?- X = 1 + 2.`

`X = 1+2`

and not `X = 3` as you might expect.

This is because `1 + 2` is a Prolog term, the functor `+` and two arguments `1` and `2`. In order to force Prolog to evaluate a term you need to use the Prolog keyword `is`.

`?- X is 1 + 2.`

`X = 3`

Some predefined arithmetic operators are:

`+` addition

`-` subtraction

`*` multiplication

`/` division

`**` power

`//` integer division

`mod` remainder after integer division

Comparison Operators

Comparison operators also force evaluation in Prolog.

The comparison operators are:

$X > Y$ X is greater than Y

$X < Y$ X is less than Y

$X \geq Y$ X is greater than or equal to Y

$X \leq Y$ X is less than or equal to Y

$X =:= Y$ X is equal to Y

$X \neq Y$ X is not equal to Y

Arithmetic (2)

Compare the output of the following 2 Prolog fragments:

`length1([], 0).`

`length2([], 0).`

`length1([_ | Tail], N) :-
 length1(Tail, N1),
 N is N1 + 1.`

`length2([_ | Tail], N) :-
 length2(Tail, N1),
 N = N1 + 1.`

`?- length1([a,b,c,d], N).
N = 4`

`?- length2([a,b,c,d], N).
N = 0+1+1+1+1`

Structures

Prolog, unlike Java and C++ structs and classes, doesn't have any inbuilt mechanisms specifically for data structures. However, we can use relations and functors to build data structures.

As we saw earlier with lists, while a list has an external representation, such as `[a,b,c,d]`, internally it is represented as:

```
.(a, .(b, .(c, .(d, []))))
```

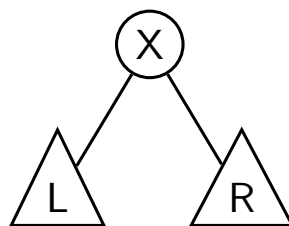
The idea is that the functor (and hence its relation) is true if the data structure exists. For example, `.(h, t)` is true if a list exists whose head is `h` and whose tail is `t`.

Binary Trees

A *binary tree* has two cases:

- an empty tree; and
- subtree that has its own root, `X`, and a left subtree, `L`, and a right subtree, `R`.

We can represent these by an atom `nil` and a functor `bt`.



```
bt (L, X, R).
```

Binary Trees (2)

To test if an element X is in a binary tree we could use the following clauses.

```
in(X, bt(_, X, _)).
```

```
in(X, bt(L, Y, _) :-  
    X ≤ Y, in(X, L).
```

```
in(X, bt(_, _, R)) :-  
    in(X, R).
```

- The 1st clause is true if the X is the root of the tree.
- The 2nd clause is invoked in the first clause fails. If X is less than or equal to the root of the tree, it searches the left subtree for X .
- The 3rd clause is invoked if X is not the root or in the left subtree. This clause searches the right subtree.
- A search of an empty tree will “fail”.

CSC1018

David Sinclair

©2024

Binary Trees (3)

Adding an element into a binary tree could be accomplished by:

```
addnode(nil, X, bt(nil, X, nil)).
```

```
addnode(bt(L, Y, R), X, bt(L1, Y, R)) :-  
    X ≤ Y, addnode(L, X, L1).
```

```
addnode(bt(L, Y, R), X, bt(L, Y, R1)) :-  
    addnode(R, X, R1).
```

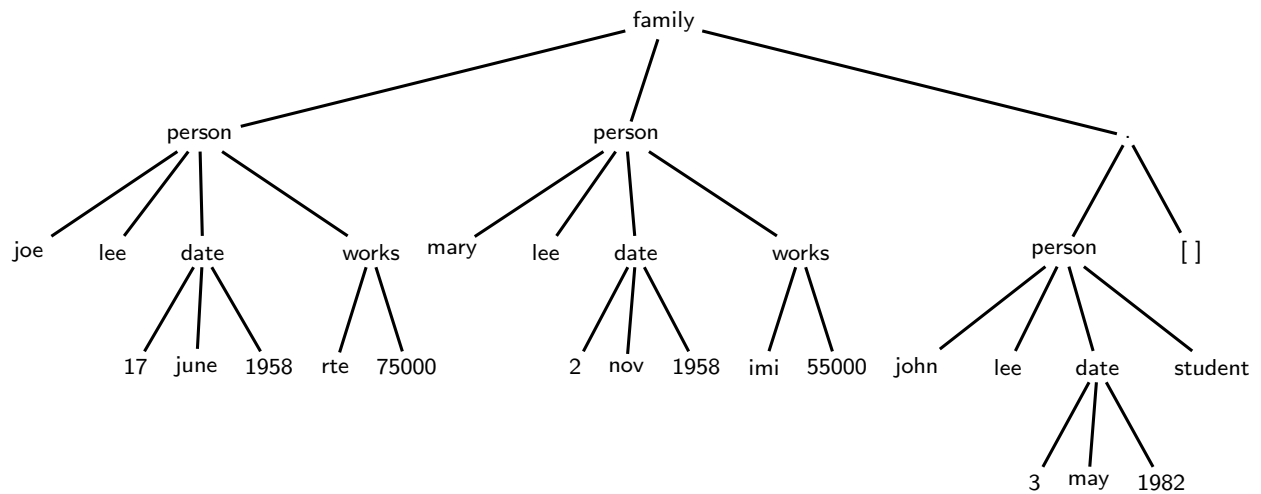
Deleting from a binary tree is a bit tricky! If it is a leaf node that is to be deleted then it is easy.

```
delleaf(T1, X, T2) :-  
    addnode(T2, X, T1).
```

If it is an internal node it requires the node that has been deleted to be replaced with something else!

Structured Data

We could model the information about a family as follows:



The `family` relation has 3 arguments, the 1st and 2nd are persons (father and mother) and the 3rd is a list of children. The `person` relation records the firstname, surname, DoB and working status of each person.

CSC1018 David Sinclair ©2024

Structured Data (2)

Here are some queries we can perform on the `family` relation.

```
father(X) :-
    family(X, -, -).
```

```
mother(X) :-
    family(-, X, -).
```

```
child(X) :-
    family(-, -, Children), member(X, Children).
```

```
exists(Person) :-
    father(Person); mother(Person); child(Person).
```

Structured Data (3)

```
dateofbirth(person(-, -, Date, -), Date).
```

```
salary(person(-, -, -, works(-, S)), S).
```

```
salary(-, 0).
```

```
after2000(X) :-
```

```
    child(X), dateofbirth(X, date(-, -, Y)), Y > 2000.
```

```
total([], 0).
```

```
total([P | L], Total) :-
```

```
    salary(P, S), total(L, Subtotal),
```

```
    Total is S + Subtotal.
```

```
?- family(F, M, C), total([F,M | C], Income).
```

Structured Data - Travel Planner

Consider a travel planner application that finds the sequence of flights between two airports, such that:

- the route starts at Place1,
- the route ends at Place2,
- all flights are on the same day of the week, and
- there is sufficient time to transfer between flights.

We will model the flight information using the `timetable` relation:

```
timetable(Place1, Place2, ListOfFlights)
```

where the `ListOfFlights` contains structured items separated by the `'/'` operator.

```
DepTime/ArrTime/FlightNumber/ListOfDays
```

Structured Data - Travel Planner (2)

An example of the `timetable` relation is:

```
timetable(dublin, london,
          [8:20/9:25/ei101/[mo, tu, th, fr],
           10:05/11:10/ba201/alldays,
           14:30/15:35/ei356/[mo, we, fr, sa]]).
```

The Prolog predicates we need are:

```
:- op(50, xfy, :).
```

```
flyday(Day, Daylist) :-
    member(Day, Daylist).
```

```
flyday(Day, alldays) :-
    member(Day, [mo, tu, we, th, fr, sa, su]).
```

Structured Data - Travel Planner (3)

```
deptime([P/P2/Fnum/Dep | _], Dep).
```

```
member(X, [X|_]).
```

```
member(X, [_|L]) :-
    member(X, L).
```

```
transfer(H1:M1, H2:M2) :-
    60 * (H2-H1)+M2-M1 >= 40.
```

```
flight(Place1, Place2, Day, Fnum, DepT, ArrT) :-
    timetable(Place1, Place2, FlightList),
    member(DepT/ArrT/Fnum/DayList, FlightList),
    flyday(Day, DayList).
```

Structured Data - Travel Planner (4)

```
% direct flight
```

```
route(P1, P2, Day, [P1/P2/Fnum/Deptime]) :-  
    flight(P1, P2, Day, Fnum, Deptime, _).
```

```
% indirect connection
```

```
route(P1, P2, Day, [(P1/P3/Fnum/Dep1)| RestRoute]) :-  
    route(P3, P2, Day, RestRoute),  
    flight(P1, P3, Day, Fnum1, Dep1, Arr1),  
    deptime(RestRoute, Dep2),  
    transfer(Arr1, Dep2).
```

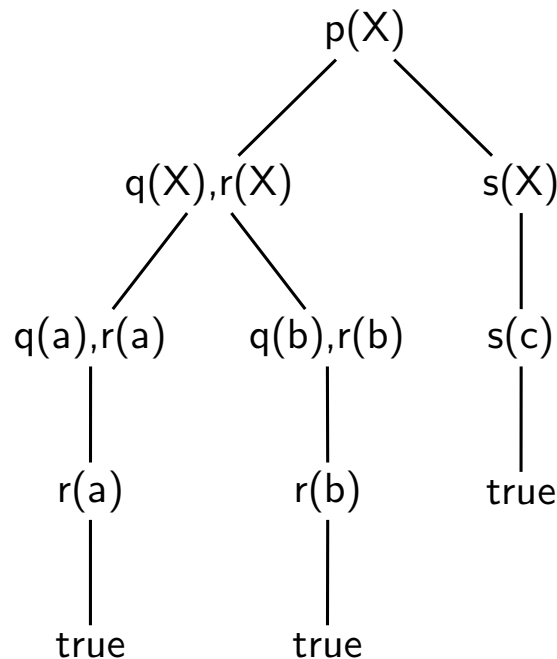
Backtracking

Prolog is a *declarative* language. The clauses in a Prolog programme describe the properties of a valid solution. The Prolog runtime searches for an instantiation of the variables that will give rise to a valid solution.

Consider:

```
p(X) :- q(X), r(X).  
p(X) :- s(X).  
q(a).  
q(b).  
r(X).  
s(c).
```

Backtracking (2)



This will return $X=a$, $X=b$ and $X=c$.

The Cut

The cut operator in Prolog, '!', control backtracking in Prolog.

- The cut operator always succeeds.
- Not only does any redo of a cut fail, the entire predicate immediately fails.

Consider

$p(t_1, \dots, t_k) :- A_1, \dots, A_m, !, B_1, \dots, B_n.$

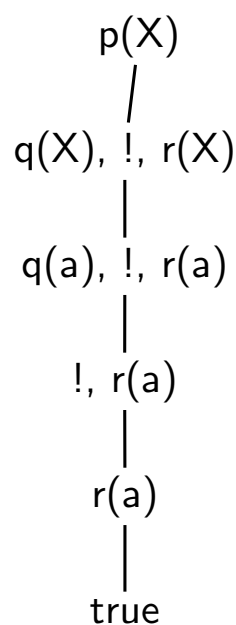
- Processing progresses as before until the cut operator.
- Once the cut is passed, all alternative calls to p are discarded.
- The current solution for literals $A_1, \dots, A_m$ is frozen and no other solutions for literals $A_1, \dots, A_m$ will be considered.

Backtracking (3)

Consider a slight modification to the previous program:

```
p(X) :- q(X), !, r(X).
p(X) :- s(X).
q(a).
q(b).
r(X).
s(c).
```

Backtracking (4)



This will return $X=a$ only.

Backtracking (5)

$p(t_1, \dots, t_k) :- A_1, \dots, A_m, !, B_1, \dots, B_n.$

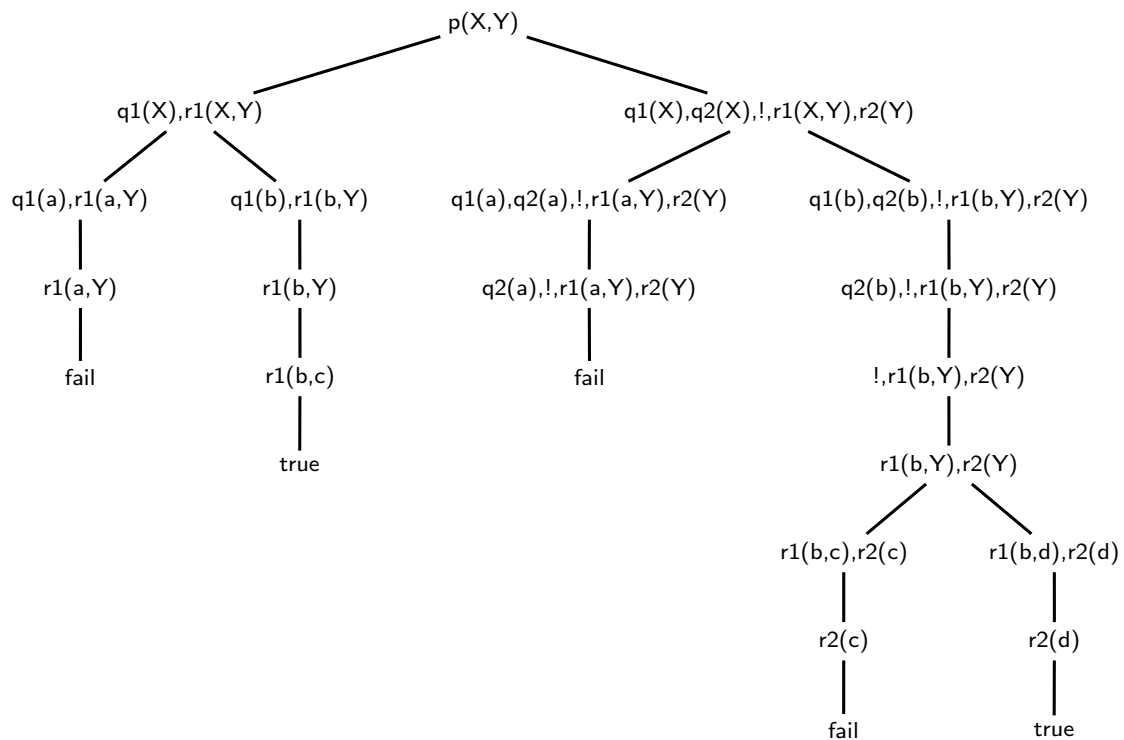
- Before a cut, backtracking can occur as normal.
 - If literal A_i fails, then alternative solutions for $A_1, \dots, A_{i-1}$ can be searched for.
- After a cut, backtracking can occur as normal.
 - If literal B_i fails, then alternative solutions for $B_1, \dots, B_{i-1}$ can be searched for.
- Backtracking **cannot** propagate through a cut.

Backtracking (6)

Consider the following program:

```
p(X,Y) :- q1(X), r1(X,Y).
p(X,Y) :- q1(X), q2(X), !, R1(X,Y), r2(Y).
p(X,Y) :- s(X,Y).
q1(a).
q1(b).
q2(b).
r1(b,c).
r1(b,d).
r2(d)
s(e,f).
```

Backtracking (7)



The solutions to $p(X,Y)$ are $X=b, Y=c$ and $X=b, Y=d$.

CSC1018 David Sinclair ©2024

Why use Cuts?

Cuts are used to:

- improve runtime by eliminating elements from the search tree (proof tree) that cannot yield any solutions,
- improve runtime by eliminating elements from the search tree (proof tree) that cannot yield any *new* solutions,
- improve on space utilisation as Prolog needs to record a lot of information on where to backtrack to and on what variable instantiations need to be tried, and
- make a nondeterministic predicate (i.e. has more than one solution) a deterministic predicate.

Examples

We can implement a max predicate as:

```
max(X,Y,X) :- X >= Y.
max(X,Y,Y) :- X < Y.
```

A slightly more efficient, and much nicer, predicate is:

```
max(X,Y,X) :- X >= Y, !.
max(X,Y,Y).
```

This is an example of an `if...then` construct. In Prolog, `if A then B` can be written as `A -> B`, which is equivalent to `A, !, B`.

Our list membership predicate from earlier is nondeterministic if an element occurs multiple times in the list (returning multiple true values).

```
member(X, [X|L]).
member(X, [_|L]) :- member(X, L).
```

Here the cut removes any alternative invocations of `add`.

CSC1018 David Sinclair ©2024

Examples (2)

Any predicate that uses the `member` predicate will then also be nondeterministic.

We can make `member` deterministic by:

```
member(X, [X|L]) :- !.
member(X, [_|L]) :- member(X, L).
```

To add an element to a list without duplication, i.e. only add an element to a list if it is not already in the list, then we can use:

```
add(X, L, L) :- member(X, L), !.
add(X, L, [X|L]).
```

Negation as failure

Prolog has a special goal called **fail** that always fails and we can use it with the cut operator to implement negation in Prolog.

We can implement a predicate called `different`, which tests if two variables do not match, as:

```
different(X,X) :- !, fail.
different(X,Y).
```

or as:

```
different(X,X) :- !, fail; true
```

We can use this combination to define a `not` predicate.

```
not(P) :-
    P, !, fail
    ;
    true.
```

Many Prolog implementations include `not` as a built-in predicate, or as a built-in goal, `\+ P`.

CSC1018

David Sinclair

©2024

Negation as failure (2)

Negation in Prolog is problematical. At times it does not correspond to our mathematical notion of negation.

A predicate is true in Prolog if the runtime systems can find an instantiation of variables that result in the conjunction of facts, defined within the Prolog programme, are true.

For example,

```
not(human(david)).
```

would return `true` if the Prolog programme did not include `human(david)` as a fact.

This is because `true` in this context means that it is not possible to prove that `human(david)` given the facts in the programme.

This is called the *closed world assumption*.

Dangers of Cut

There are a few dangers that we need to be aware of when using cuts:

- There are two type of cuts,
 - *green cuts* that do not change the declarative meaning of the program; and
 - *red cuts* that break the link between the declarative and procedural meanings of a program.

Consider the following examples:

$p :- a, b.$
 $p :- c.$

This is $p \iff (a \wedge b) \vee c.$

Changing the order of the clauses would have no effect.

Dangers of Cut (2)

Now let's add a cut!

$p :- a, !, b.$
 $p :- c.$

Now the meaning has changed to $p \iff (a \wedge b) \vee (\neg a \wedge c).$

If we swap the clauses we get a program that has a meaning equivalent to the original “non-cut” program.

$p :- c.$
 $p :- a, !, b.$

Which means $p \iff c \vee (a \wedge b).$

- *cut & fail* is not the same as mathematical negation!
- The negation of a goal with existentially quantified variable is not “does there exist a value of the variable for which the goal is not true”!

Input/Output

Prolog communicates with input and output devices via *streams*.

At any given time only one input stream is active and only one output stream is active. Initially both of these streams are the *user terminal*, *user*.

To change the current input stream to *filename*, we use the *see* predicate.

see(*filename*).

To change the current output stream to *filename*, we use the *tell* predicate.

tell(*filename*).

The goals *seen* and *told* close the current input and output streams.

Input/Output (2)

The built-in predicate *read* reads a term from the current input stream.

read(*X*).

This will cause the next term *T* to be read and matched against *X*. If the matching succeeds, *X* will be instantiated as *T*.

read(*x*) is deterministic so if the matching fails, there is no backtracking.

Each term in the input must be followed by a space, carriage return or a period. If the end of the file is reached then *X* will be instantiated by the atom *end_of_file*.

Input/Output (3)

The built-in predicate `write` writes a term to the current input stream.

`write(X).`

`X` will be displayed in standard syntactic form.

The built-in predicates `tab` and `nl` can be used to format the output.

`tab(N).`

Write N spaces to the current output stream.

`nl.`

Starts a new line in the current output stream.

Input/Output (4)

A typical Prolog “conversation” would be:

`cube :-`

```
write('Next item: '),
read(X),
process(X).
```

`process(stop) :- !.`

`process(N) :-`

```
C is N*N*N,
write('Cube of '), write(N),
write(' is '), write(C), nl.
```

Input/Output (5)

A typical goal sequence when processing files would be:

```
..., see(F), processfile, see(user), ...
```

```
processfile :-  
    read(X),  
    process(X).
```

```
process(end_of_file) :- !.
```

```
process(X) :-  
    treat(X),  
    processfile.
```

where `treat(X)` performs the required processing on `X`.

Input/Output (6)

`put(C)`

outputs the character whose ASCII value corresponds to `C`.

`get0(C)`

reads a single character from the input stream and instantiates `C` to its ASCII value.

`get(C)`

is similar to `get0` but skips over all non-printable characters.

Constructing and Decomposing Atoms

At times it is useful to compose atoms out of characters read in from the input stream, or to decompose an atom into a sequence of characters. We can do this with the `name` predicate.

`name(A, CL)`

which is true if `CL` is the list of ASCII codes corresponding to the characters in the atom `A`.

`name(david, CL)`.

will instantiate `CL` as `[101,97,118,105,101]` and

`name(A, [101,111,104])`.

will instantiate `A` as `[dog]`.

Testing Terms

The following built-in predicates are used to test the type of various terms.

`var(X)` true if `X` is an uninstantiated variable.

`nonvar(X)` true if `X` is not a variable or if it is an already instantiated variable.

`atom(X)` true if `X` represents an atom.

`integer(X)` true if `X` represents an integer.

`float(X)` true if `X` represents a floating point number.

`number(X)` true if `X` represents a number.

`atomic(X)` true if `X` represents a number or an atom.

`compound(X)` true if `X` represents a compound term (i.e. a structure).

Testing Terms (2)

The following example is an incorrect attempt at counting the number of occurrences of an atom in a list.

```
count( _ , [ ] , 0 ).
```

```
count(A, [A|L], N) :-  
    count(A, L, N1), N is N1 + 1.
```

```
count(A, [_|L], N) :-  
    count(A, L, N).
```

Here is why!

```
?- count(a, [a,b,X,Y], N).  
N = 3.  
?- count(b, [a,b,X,Y], N).  
N = 3.  
?- L=[a,b,X,Y], count(a, L, Na), count(b, L, Nb).  
Na = 3  
Nb = 1  
X = a
```

CSC1018 David Sinclair ©2024

Testing Terms (3)

A correct implementation is:

```
count( _ , [ ] , 0 ).
```

```
count(A, [B|L], N) :-  
    atom(B), A = B, !,      % is B the atom A?  
    count(A, L, N1),        % count occurrences of A in tail  
    N is N1 + 1.  
;  
count(A, L, N).             % else count occurrences of A  
                             % in tail
```

Constructing and Decomposing Terms

The goal

$\text{Term} =.. L$

is true if L is a list that contains the principal functor of Term followed by its arguments.

```
?- f(a,b) =.. L.
L = [f,a,b].
```

```
?- T =.. [rectangle,3,5].
T = rectangle(3,5).
```

Suppose we wish to manipulate an expression by substituting all occurrences of one subterm for another subterm, e.g.

```
?- substitute(sin(x), 2*sin(x)*f(sin(x)), t, F).
F = 2*t*f(t)
```

Constructing and Decomposing Terms (2)

% Substituting a whole term

```
substitute(Term, Term, Term1, Term1) :- !.
```

% Substituting when the expression is atomic

```
substitute(_, Exp, _, Exp) :-
    atomic(Exp), !.
```

% Otherwise

```
substitute(Term, Exp, Term1, Exp1) :-
    Exp =.. [F|Args],                % get arguments
    sublist(Term, Args, Term1, Args1), % perform substitution
                                       % on them
    Exp1 =.. [F|Args1].
```

```
sublist(_, [], _, []).
```

```
sublist(Term, [Exp|Exps], Term1, [Exp1|Exps1]) :-
    substitute(Term, Exp, Term1, Exp1),
    sublist(Term, Exps, Term1, Exps1).
```

Constructing and Decomposing Terms (3)

One of the advantages of the `=..` predicate is that it can be used to construct goals.

Suppose `obtain` and `compute` are user-defined predicates that get the functor and arguments for some goal (possibly from a file):

```
obtain(Functor),
compute(Arglist),
Goal =.. [Functor|Arglist],
Goal.
```

Some Prolog implementations require that a goal is either an atom or its principal functor is an atom. In these cases we can use the `call` predicate.

```
obtain(Functor),
compute(Arglist),
Goal =.. [Functor|Arglist],
call(Goal).
```

functor and arg

`functor` and `arg` are additional built-in predicates for manipulating terms.

```
functor(Term, F, N)
```

is true if `F` is the principal functor of `Term` and the arity of `F` is `N`.

```
?- functor(t(f(X),X,t),F,N).
F = t
N = 3
```

```
arg(N,Term,A)
```

is true if `A` is the `N`th argument in `Term`.

```
?- arg(2,f(X,t(a),t(b)),Y).
Y = t(a)
```

functor and arg (2)

A typical usage of `functor` and `arg` is to generate a term and instantiate its arguments.

```
?- functor(D, date, 3),
   arg(1, D, 29),
   arg(2, D, feb),
   arg(3, D, 2012).
```

```
D = date(29, feb, 2012)
```

Equality and Comparison Operators

We have already seen the following equality operators.

<code>X = Y</code>	X matches Y
<code>X is E</code>	X matches the value of the arithmetic expression E
<code>E1 == E2</code>	The value of the arithmetic expression E1 matches the value of the arithmetic expression E2
<code>E1 \= E2</code>	The value of the arithmetic expression E1 does not match the value of the arithmetic expression E2

Another form of equality is *literal equality*. with operator `==`.

```
T1 == T2
```

is true if term T1 is identical to term T2. T1 and T2 have to have the exact same structure and corresponding components must also be the same, including the name of variables.

Equality and Comparison Operators (2)

For example,

?- $f(a, b) = f(a, X)$.
no.

?- $f(a, X) = f(a, Y)$.
no.

?- $f(a, X) = f(a, X)$.
yes.

Corresponding to $<$, $=$, $>$ and $>=$, which compare terms arithmetically, are the operators $@<$, $@=$, $@>$ and $@>=$, which compare terms alphabetically. These operators can be used to define an ordering among terms. If the principal functors are the same, then the top-most, left-most functors decide.

Database manipulation

During the execution of a Prolog program, the database of clauses can be modified using the `assert`, `asserta`, `assertz` and `retract` predicates.

The goal

`assert(C)`

always succeeds and adds the clause `C` to the database. `assert`, and its pseudonym `assertz`, add the clause to the end of the database. `asserta` adds the clause to the start of the database.

The goal

`retract(C)`

always succeeds and removes the clause `C` to the database. `retract` is non-deterministic, and via backtracking, it can remove a whole set of clauses.

`retract(p(X)).`

Repeat

Repeat is a goal that always succeeds and is non-deterministic and when it is reached generates another execution branch.

```
dosquares :-
    repeat ,
    read(X) ,
    (
    X=stop , !
    ;
    Y is X*X, write(Y) ,
    fail
    ).
```

bagof, setof and findall

These predicates are useful in generating lists of objects that satisfy some goal.

```
bagof(X,P,L)
```

will generate so list L of objects X that satisfies the goal P.

Consider the following facts

```
age(peter , 7).
age(ann , 5).
age(pat , 8).
age(tom , 5).
```

Then,

```
?- bagof(Child , age(Child , 5), L).
L = [ann , tom].
```

gives a list of all children whose age is 5.

bagof, setof and findall (2)

If we did not care about the value of their age, we can use the $\wedge$ construct which just checks if its left operand exists.

```
?- bagof(Child , Age  $\wedge$  age(Child , Age) , L).
L = [peter , ann , pat , tom].
```

The setof predicate is similar to bagof but duplicates are eliminated.

```
setof(X,P,L)
```

will generate so list L of objects X, with no duplicates, that satisfies the goal P. The ordering of objects is defined by '@<'

findall is similar to bagof except that all of the objects X are collected regardless of different instantiations for the variables in P that are not shared with X.

```
?- findall(Child , age(Child , Age) , L).
L = [peter , ann , pat , tom].
```

Efficient Prolog Programming

Some general guidelines for Prolog programming:

- Use recursion:
 - Base case (boundary case, trivial case)
 - Inductive case (general case) where the solution is based on the solution of a "smaller" problem.

Consider a predicate `maplist` that is true if the 3rd argument is a the list in the 1st argument with the function in the 2nd argument applied to each element of the list.

- Base case is the empty list.
- Inductive case is the predicate applied to the head of the list concatenated with `maplist` applied to the tail of the list.

```
maplist([],_, []).
maplist([X|Tail], P, [NewX|NewTail]) :-
    G =.. [P, X, NewX], call(G),
    maplist(Tail, P, NewTail.)
```

```
squares(X, Y) :-
    Y is X * X.
?- maplist([1,2,3,4], squares, L).
L = [1,4,9,16].
```

Efficient Prolog Programming(2)

- Generalise:
 - Generalising a problem often lets you find a recursive solution.

Consider the *eight queens problem*, i.e. how do you place eight queens on a chessboard so that no queen attacks another queen.

We can generalise it to an *n-queen problem* with a predicate `nqueen(Pos, N)` and then solve it recursively.

- Base case: $N = 0$ is trivially true.
- General case: $N > 0$
 - Find a solution for $(N-1)$ queens problem.
 - Add remaining queen so it does not attack any other queen.

```
eightqueen(Pos) :- nqueen(Pos, 8).
```

Efficient Prolog Programming(3)

- Use pictures:
 - Prolog is suited to problems in which there are relations between objects. This can be easily drawn and this usually helps identify the essential relations.
 - Structured data items are usually visualised as trees
 - Because Prolog is declarative, order is not important and hence there are less restrictions on how a problem can be visualised.
- Improve search efficiency by using cuts to stop the execution of useless alternatives.
 - Be careful with cuts. *Green cuts* do not change the meaning of a Prolog program, but *red cuts* will!
- Use suitable data structures.

Style

The following guidelines don't guarantee that a Prolog program is correct but does make a Prolog program is easier to understand, modify and debug.

- Program clauses should be short and typically should contain only a few goals.
- Procedures should be short unless it has a uniform structure.
- Procedure and variable names should be meaningful.
- Use indentation, spacing and blank to improve the layout the program **but** be consistent.
- Conventions about naming and layout can be vary from person to person, such as `ShoppingList` or `Shopping_list`. Which stylistic convention is used is not important as long as it is used **consistently**.
- Be careful using cuts and anything that implicitly uses them, such as `not`.

CSC1018

David Sinclair

©2024

Style (2)

- `assert` and `retract` modify the actual program and can make a program difficult to understand and debug.
- Semicolons can make a program difficult to understand. Don't be afraid to use "extra" lines.

```
merge(List1, List2, List3) :-
    List1 = [], !, List3 = List2; % 1st list empty
    List2 = [], List3 = List1;    % 2nd list empty
    List1 = [X|Rest1], List2 = [Y|Rest2],
    ( X < Y, !,
      Z = X,
      merge(Rest1, List2, Rest2);
      Z = Y,
      merge(List1, Rest2, Rest3)),
    List3 = [Z|Rest3].
```

is not as clear as

CSC1018

David Sinclair

©2024

Style (3)

```
merge([], List, List) :-  
    !.           % prevent redundant solutions  
  
merge(List, [], List).  
  
merge([X|Rest1], [Y|Rest2], [X|Rest3]) :-  
    X < Y, !,  
    merge(Rest1, [Y|Rest2], Rest3).  
  
merge(List1, [Y|Rest2], [Y|Rest3]) :-  
    merge(List1, Rest2, Rest3).
```

Style (4)

- **Use comments!** But don't over use them.
 - Describe what the program does.
 - Describe top-level predicates.
 - Describe how the main concepts are represented as structured data.
 - What is the meaning of each predicate?
 - What are its arguments and which are input arguments and which are output arguments?
 - Describe any algorithmic or implementation details.
 - What are the limitations of the program?

Debugging

As with any language, the secret to debugging is not to debug the whole program, but to debug and test small parts of the program and then use these “trusted” elements to build larger elements.

Prolog has a couple of features that aid debugging:

- Prolog is an interactive language and hence parts of the program can be invoked by submitting suitable queries to the Prolog system.
- Prolog has built-in predicates that enable and disable *tracing* at different levels. While *tracing* the following information is displayed.
 - The predicate name and argument values when a goal is invoked.
 - When a goal succeeds, the values of the arguments that satisfy the goal.
 - Re-entry information during backtracking.

Debugging (2)

trace

is a predicate that enables exhaustive tracing.

notrace

is a predicate that disables further tracing.

spy(P)

is a predicate that enables tracing on a specific goal P.

nospy(P)

is a predicate that disables tracing on a specific goal P.

Accumulators

Each recursive invocation of a procedure takes up memory space. However there is a special form of recursion called *tail recursion* that most Prolog systems can implement as iteration. This implementation of *tail recursion* as iteration is called *tail recursion optimisation* or *last call optimisation*.

In order for a procedure to be *tail recursive*:

- the recursive call to the procedure must be the last clause; and
- the procedure must be deterministic so that not backtracking occurs after the last recursive call (we can force this behaviour by inserting a cut before the last clause).

Accumulators (2)

The following procedure, that sums the values in a list, is not tail recursive.

```
sumlist([], 0).
```

```
sumlist([X|Tail], Sum) :-
    sumlist(Tail, TailSum),
    Sum is X + TailSum.
```

We can use a trick called *accumulators* to transform it into a tail recursive procedure.

```
sumlist(List, Sum) :-
    sumlist(List, 0, Sum).
```

```
sumlist([], Sum, Sum).
```

```
sumlist([X|Tail], PartialSum, Sum) :-
    NewPartialSum is PartialSum + X,
    sumlist(Tail, NewPartialSum, Sum).
```

Accumulators (3)

Aside: `sumlist` with 2 arguments (`sumlist/2`) is a different procedure than `sumlist` with 3 arguments (`sumlist/3`).

A naïve implementation of a string reversal procedure is:

```
reverse([], []).
```

```
reverse([X|Tail], RevList) :-  
    reverse(Tail, RevTail),  
    conc(RevTail, [X], RevList).  % append X to RevTail
```

A more efficient implementation (why?) is:

```
reverse(List, RevList) :-  
    reverse(List, [], RevList).
```

```
reverse([], RevList, RevList).
```

```
reverse([X|Tail], Acc, RevList) :-  
    reverse(Tail, [X|Acc], RevTail).
```